

PROJEKT ENGINEERING

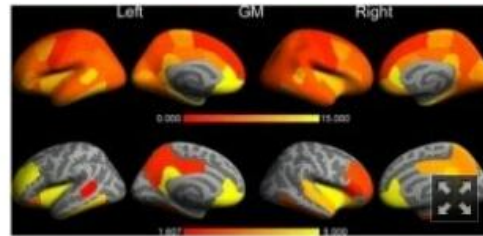
Testen

Herwig Mayr

Fakultät für Informatik, Kommunikation und Medien
Fachhochschule OÖ, Hagenberg

Macs und PCs liefern unterschiedlich aussehende Gehirnschans

 Oliver Schwab Dienstag 19.06.2012



Ein Forscherteam aus Maastricht und Limburg hat festgestellt, dass eine wichtige Analysesoftware für medizinische Bilder des Gehirns unterschiedliche Ergebnisse liefert – je nachdem ob sie auf einem Mac oder einem PC läuft.

Das Team analysierte die Daten von 30 Gehirnschans mit dem Programmpaket FreeSurfer, womit in der Regel die Größe verschiedener Hirnbereiche gemessen wird.

Die Software wurde auf PCs und verschiedenen Mac OS Versionen getestet. In Bezug auf die Größe verschiedener Hirnregionen kamen nicht nur zwischen PC und Mac, sondern auch zwischen unterschiedlichen Mac OS Versionen andere Ergebnisse heraus.

Diese Unterschiede bewegen sich in einer Spanne von 2 bis 5 Prozent, im parahippocampalen und entorhinalen Cortex lagen die Ergebnisse sogar bis zu 15 Prozent auseinander.

Warum dieses Problem auftritt, ist leider nicht bekannt. Es zeigt aber, dass unterschiedliche Krankenhäuser aufgrund unterschiedlicher Ergebnisse unterschiedliche Behandlungsmethoden beschließen könnten. Und ein Krankenhaus nach einem Update des Betriebssystems ebenso. [PloS One via NeuroSkeptic]

(Bericht: gizmodo.de)



Testen Ziel

Testen – Ziel

Testen = systematisches Aufzeigen von Fehlern
in minimaler Zeit und
mit minimalem Aufwand

Man kann nur die Anwesenheit von Fehlern zeigen;
Fehlerfreiheit lässt sich niemals zeigen! (Dijkstra)

- > Jedes Programm enthält Fehler („mentale Irrtumsrate“, Halstead).
- > Forderung nach „100% Fehlerfreiheit“ demotiviert Programmierer!



(Grafik: www.mi.fu-berlin.de)

Testen – Ziele aus Stakeholdersicht

Entwicklersicht

Testen ist (aus Programmiersicht) ein destruktiver Prozess!
-> Testen ist psychologisch negativ besetzt.

Kundensicht

Ein erfolgreicher Test deckt einen Fehler auf, bevor ihn der Kunde findet!
-> Aus gesamter Projektsicht ist Testen konstruktiv!



Beispiel/Übung: Wie kann man Tester motivieren zu testen?

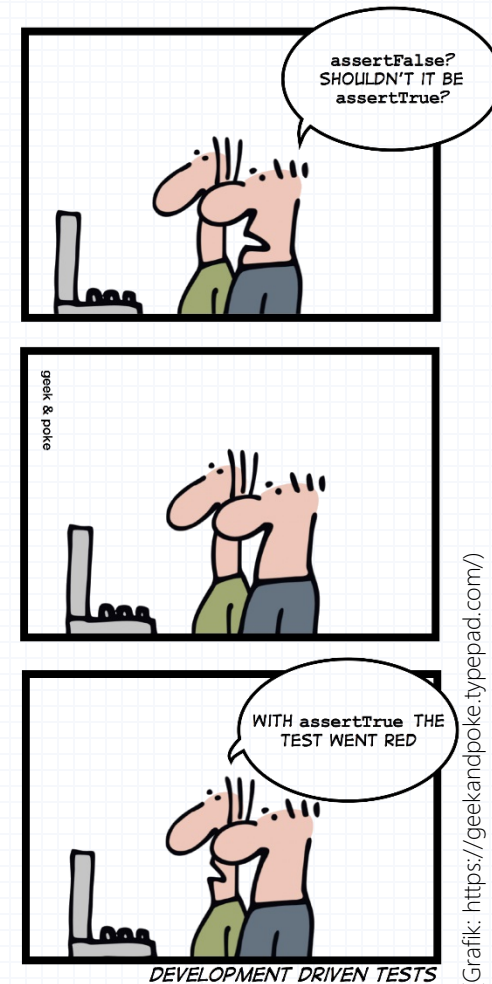
Testen – Rollenverteilung

Der Implementierer soll nicht testen!

- Aufgabe ist Fehler zu finden.
- Implementierer ist zu wenig destruktiv.
- Implementierer wiederholt konzeptuelle Fehler beim Testen.
- Andere Testperson motiviert Implementierer.
- Implementierer dokumentiert gefundene Fehler nicht.

Der Tester soll nicht korrigieren!

- Tester konzentriert sich auf extern beobachtbares Verhalten.
- Tester kennt Code (oft) nicht.
- Fehlerkorrektur hindert die Fehlersuche.
- Fehlerkorrektur bedeutet Mehrarbeit und senkt Fehlerentdeckungsrate.



Testen Aufgaben

Testen – Aufgaben

Erkennen von Analyse- und Designfehlern

- Testen „von außen“ bedeutet intensives Arbeiten mit der Benutzerschnittstelle.
- Inkonsistenzen und fehlerhafte Bedienungsabläufe sind oft Analysefehler oder Designfehler!

Erkennen von Implementierungsfehlern

- strukturiert suchen (z.B. Versuch alle Fehlermeldungen zu generieren, Testen von Kompatibilität)

Bestimmen des Produktstatus

- Alphatesten, Betatesten



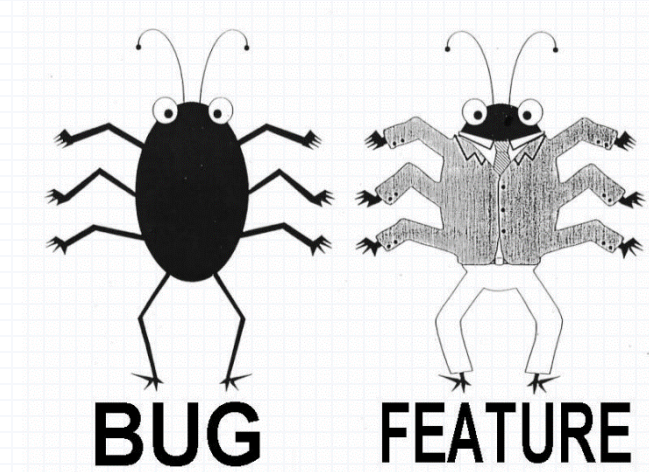
(Grafik: c't)

Aufgaben – Bestimmen des Produktstatus

Alphatesten

Testen der einzelnen Funktionalitäten bottom-up

Produkt als Gesamtes noch eher instabil (**Alphaversion**), Testen einzelner funktionaler Anforderungen ist jedoch bereits möglich



(Grafik: stackoverflow.com)

Betatesten

Testen des Produkts als Gesamtes anhand realer Beispielabläufe (**Betaversion**; nicht unbedingt volle Funktionalität)

- Mitarbeiter des **Auftraggebers** oder speziell eingestellte Betatester fungieren als „Testpiloten“ -> vorsichtiges Arbeiten, Kooperation mit Entwicklern
- Projektgruppe des **Auftragnehmers** stellt Betatester ab, Auftraggeber liefert geeignete Beispiele -> Vorteile beim Erklären und Beheben von Fehlern, Nachteile beim Einschätzen der Schwere von Fehlern bzw. was als Fehler gesehen wird

Beispiel/Übung: Mögliche Alpha- und Betatests im LEGO-Projekt

- Alphatests:

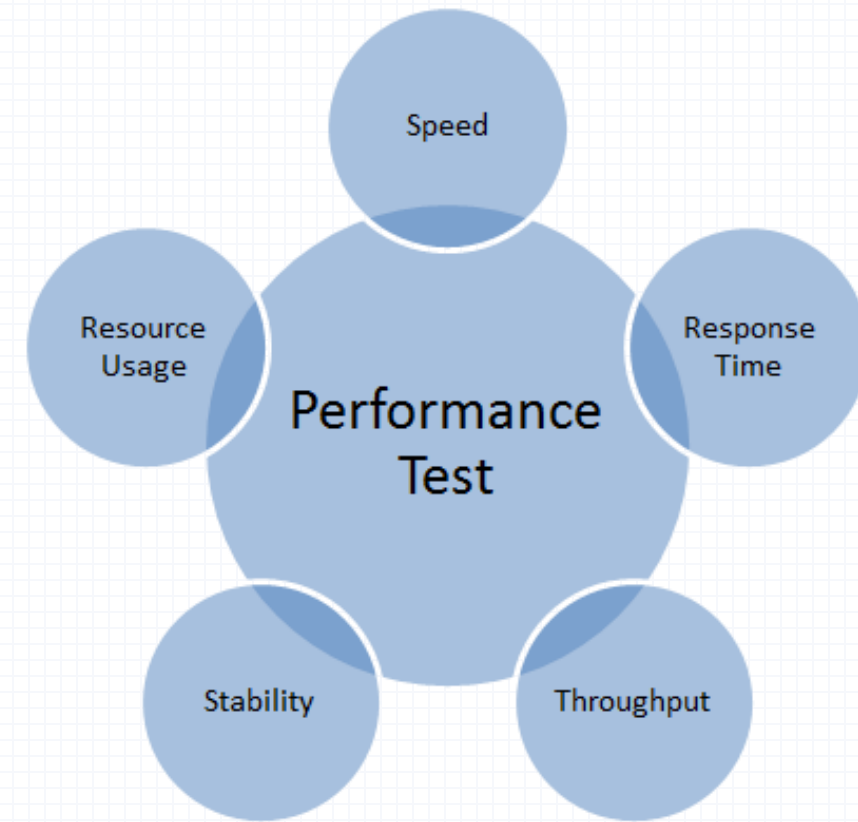
- Betatests:

Aufgaben – Testen der Produktperformanz

Performanztest – Einsatz teilweise bereits während des Debuggens, während des Testens aber aus Anwendersicht

Arten:

- **Laufzeittest** testet Laufzeitverhalten als Vorgabe für notwendiges Tunen.
- **Speichertest** testet die Speicheranforderungen.
- **Volumentest** testet maximal bewältigbaren Aufgabenumfang.
- **Belastungstest** testet Stabilität bei hoher Systemaktivität.
- **Benchmarkvergleich** vergleicht Produktversionen.



(Grafik: telerikhelper.net)

Aufgaben – Vorbereiten der Abnahme

Abnahmevortest

nimmt die Abnahmesituation beim Auftraggeber vorweg
(Test der **Abnahmesuite**)

Abnahmesuite wird **idealerweise vom Auftraggeber erstellt**; in der Praxis oft gemeinsam mit dem Auftragnehmer



(Grafik: blog.solidcraft.eu)

Testen Techniken

Testen – Techniken

Techniken

- Testfalldesign, Testsuitedesign
- Generierung und Auswahl von Testfällen
- Testablaufplanung
- Verifikation und Validierung
- Regressionstesten
- Testautomatisierung

Techniken – Testfalldesign

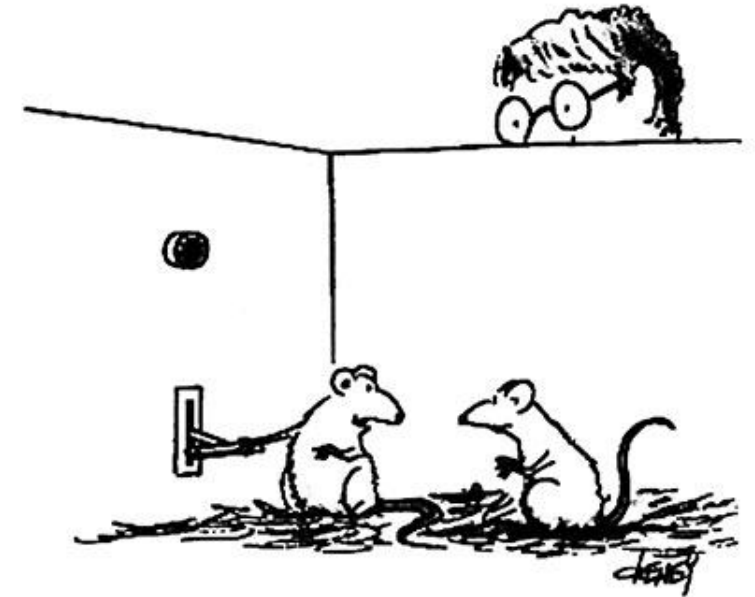
Ein guter Testfall

- macht Programmfehler offensichtlich,
- findet wahrscheinlich einen Fehler,
- ist weder zu einfach noch zu komplex.

Testprozess soll möglichst systematisch sein!

Gängigste Techniken:

- Äquivalenzklassen bilden
- Grenzwerte analysieren
- Ursache-Wirkungs-Zusammenhänge analysieren



Pass auf, es ist ein wirklich interessantes Phänomen. Immer wenn ich diesen Hebel umlege, seufzt der Postdok zufrieden auf.

(Grafik: www.labjournal.de)

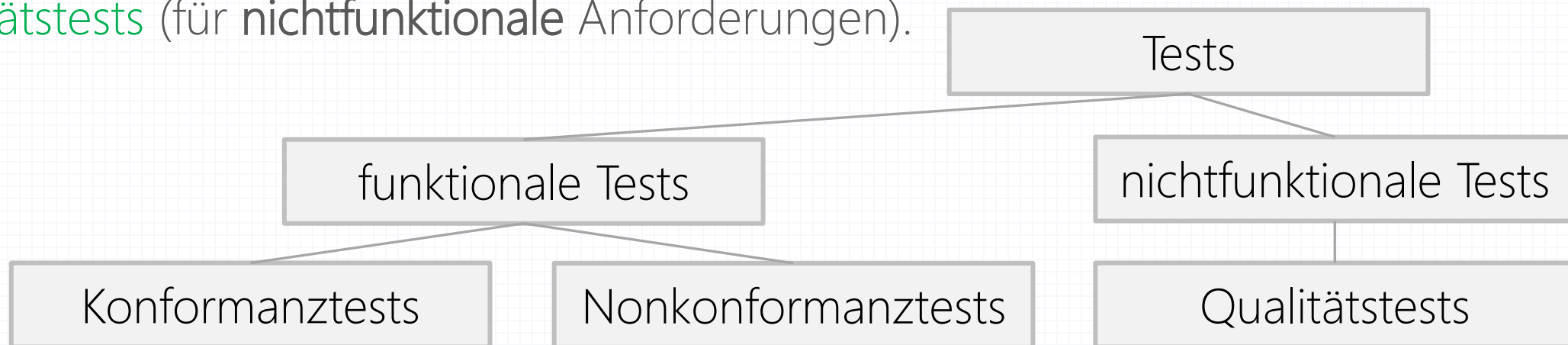
Techniken – Testsuitedesign

Erstellen eines Verzeichnisses geeigneter Testfälle ("Testfallsammlung")

Man kann nie alle Anwendungsmöglichkeiten eines Systems testen.

Gliederung in

- Konformanztests (müssen korrektes Ergebnis liefern),
- Nonkonformanztests (müssen korrekte Fehlermeldung liefern) und
- Qualitätstests (für nichtfunktionale Anforderungen).



Beispiel/Übung: Testfälle verschiedener Testklassen im LEGO-Projekt

- Konformanztests:
- Nonkonformanztests:
- Qualitätstests:

Techniken – Testfallgenerierung

Vorgehensweisen:

- aus Pflichtenheft und Dokumentation extrahieren und Grenzwerte für die Eingabewerte festlegen
- mit Referenzprogramm korrekte Testergebnisse erstellen
- interaktiv mit System arbeiten
- Konkurrenzprodukte zu Vergleichszwecken heranziehen
- korrekte Ergebnisse für spätere Vergleiche aufheben
- alle Ergebnisse immer elektronisch aufheben



Techniken – Testfallauswahl

Kriterien:

1. „Fehler leben in Kolonien“ (Myers)
-> **Fehler nahe bei Fehlern** suchen
2. für Anwender **verheerende** oder auffällige **Fehler** suchen
3. am **häufigsten verwendete** Programmbereiche ermitteln
4. wesentliche **Produktvorteile** sicher stellen
5. für die Implementierer **schwierigste Bereiche** ermitteln
6. für den Tester **verständlichste Bereiche** auswählen



(Bild: www.reflex.hu)

Techniken – Testablaufplanung

beginnt spätestens mit Verfügbarkeit des Pflichtenhefts

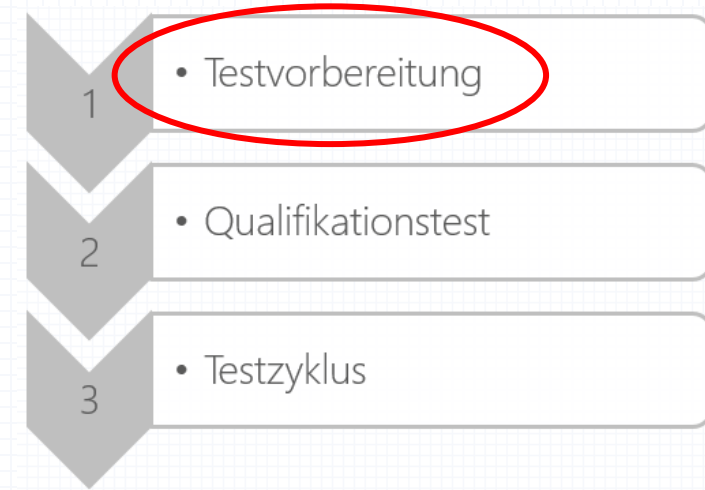


Testablaufplanung – Testvorbereitung

Beginn während des Designs, spätestens während der Implementierung

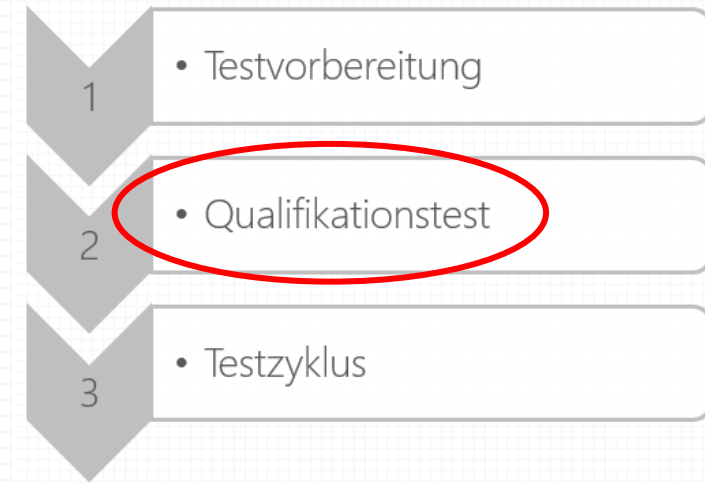
Aktivitäten ohne verfügbares Testprodukt:

- Ziel und Aufgaben des Projekts lesen
- Review existierender Dokumente durchführen
- Testdaten des Auftraggebers anfordern und analysieren
- Abnahmeprozess festlegen
- Benutzerschnittstelle und -dokumentation reviewen
- Konkurrenzprodukte analysieren



Testablaufplanung – Qualifikationstest

prüfen, ob eine Programmversion stabil genug fürs Testen ist -> Erstellen einer **Qualifikationstestsuite**,
Testen im **Qualifikationstest**



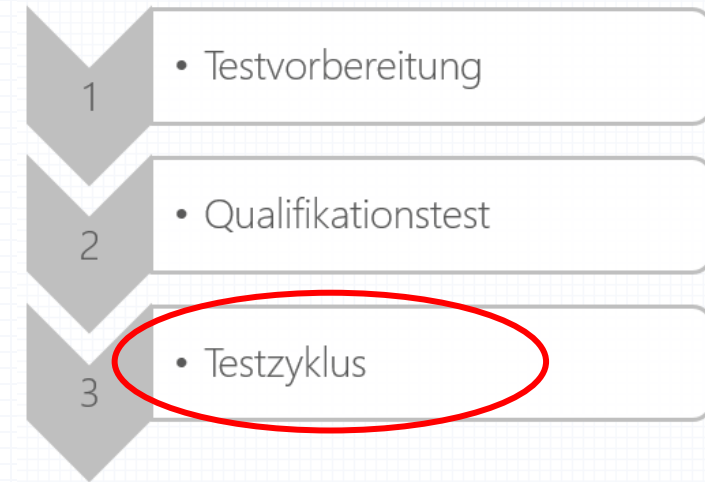
Vorgehen:

- Qualifikationstest soll nur Hauptbereiche testen.
- Sein Sinn muss den Implementierern klar sein.
- Eine Automatisierung dieses Tests ist sehr sinnvoll.
- Qualifikationstestsuite evtl. Implementierern zugänglich machen.

Testablaufplanung – Testzyklus

Schritte:

1. „Start with a bang!“ – Implementierer rasch mit Fehlerbehebungsaufträgen „versorgen“
2. Produktstabilität und -zuverlässigkeit abschätzen (Prognose für notwendige Testzyklen)
3. Offensichtliche Fehler aufdecken (Simulation des ersten Arbeitstages beim Anwender, Inkonsistenzen zur Dokumentation)
4. Testsuite(n) durchlaufen
5. Ergebnisse dokumentieren (möglichst elektronisch und geeignet für automatische Wiederholung/Prüfung)
6. Testsuite(n) adaptieren



Inkrementelles Testen ist Big-Bang-Testen auf jeden Fall vorzuziehen!

Techniken – Regressionstesten

auch: Retesten

Zweck:

- Überprüfung der Fehlerkorrektur
- Absicherung gegen neu eingeschleppte Fehler

mittels **Regressionstestsuite**, die laufend (möglichst automatisch) abgetestet und adaptiert wird

Regressionstestsuite soll möglichst klein gehalten werden (redundante Tests entfernen, Tests kombinieren).

Techniken – Testautomatisierung (I)

Manuelles Testen ist fehleranfällig -> müsste selbst laufend getestet werden!

Daher **Testautomatisierung**:

1. Testfälle erzeugen (-> Programmverifikation!)
2. Testfälle durchlaufen
3. Ergebnisse überprüfen
4. Testresultate visualisieren
5. Testresultate protokollieren

kommerzielle Beispiele:



Techniken – Testautomatisierung (II)

Testautomatisierung benötigt Testrahmensysteme für Systemtests

-> oftmals hohe Amortisationskosten und lange Amortisationszeit

weitere Risiken:

- zu viele simple Testfälle
- wenig neue Testfälle
- Einlernaufwand unterschätzt
- Ergebnisse schlecht analysiert

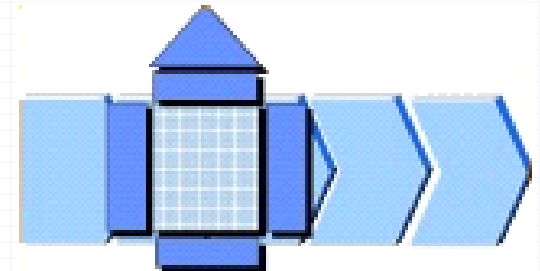


(Grafik: www.jancornelius.de)

Einschub: Verifikation vs. Validierung („V & V“)

V & V – Orientierung an Kundenwünschen / Anforderungen

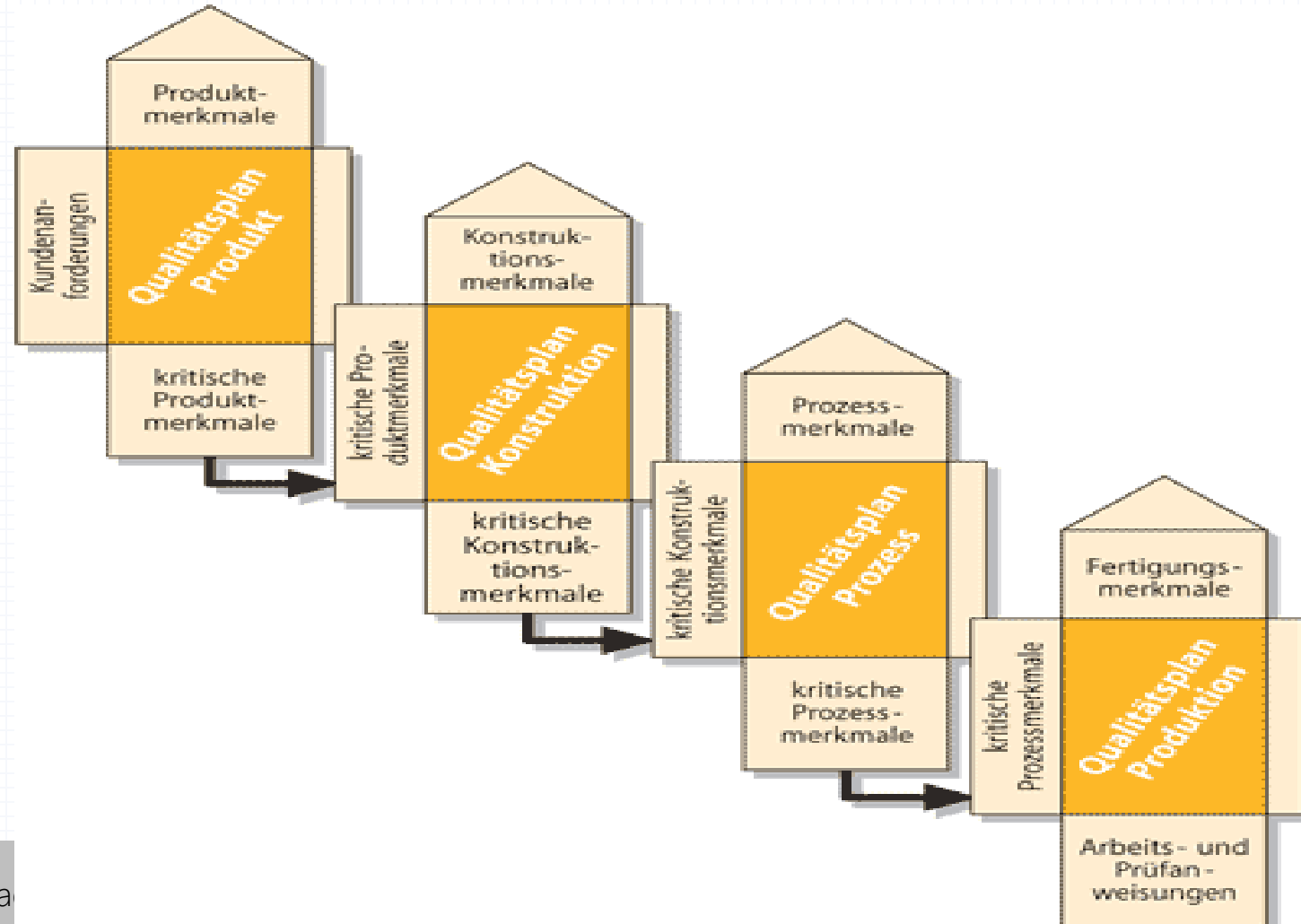
- Was der Kunde sagt (**Wünsche**) ≠ was der Kunde wirklich braucht (**Anforderungen**)!
- Der Kunde sagt nicht alle Anforderungen (**explizite Anforderungen** versus **implizite Anforderungen**)!
- Eine **systematische Erhebung** der Wünsche/Anforderungen inkl. Bewertung ist nötig.
- Die Erhebung erfolgt z.B. durch „**Quality Function Deployment**“ (**QFD**): Was/Wie-Trennung entsprechend Funktionalität/Merkmal $\Rightarrow$ „**House of Quality**“ (**HoQ**). [J. Akao]



V & V – Quality Function Deployment (QFD) (I)

Deutsch in etwa: Qualitätsfunktionen-Darstellung

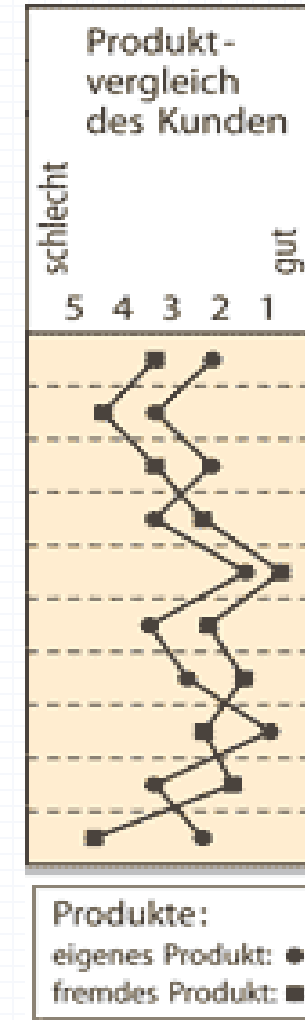
Vier Phasen:



V & V – Quality Function Deployment (QFD) (II)

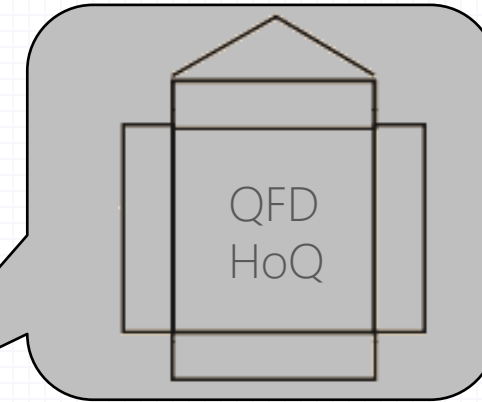
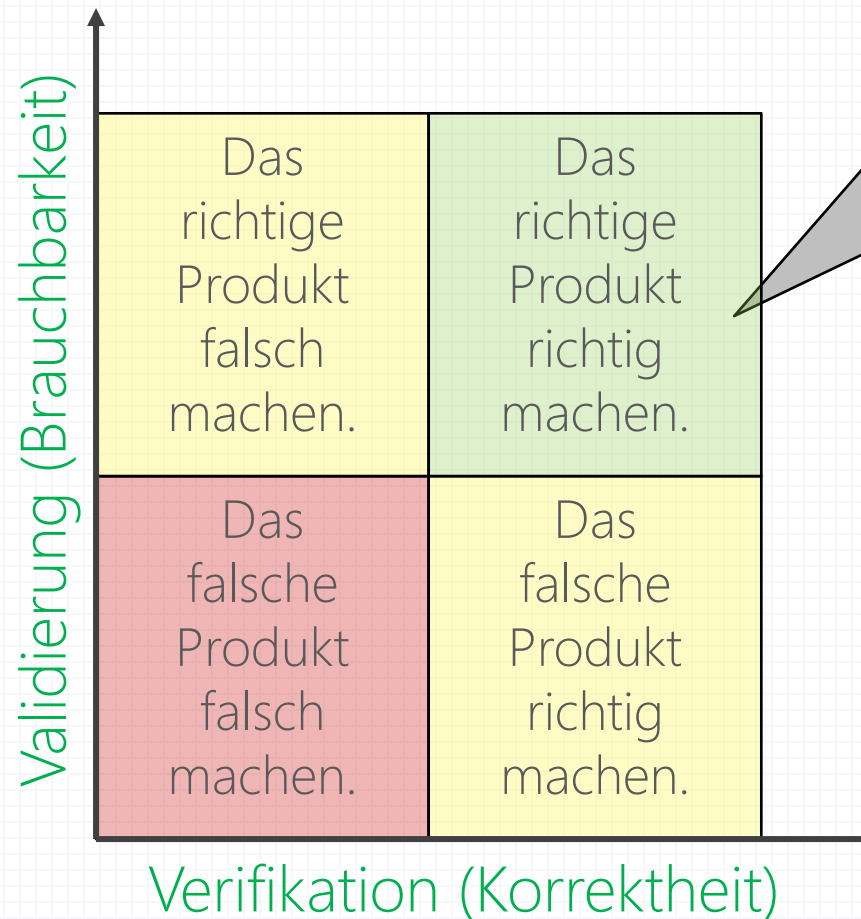
Schrittweises Vorgehen:

1. Kundenanforderungen ermitteln
2. Kundenanforderungen bewerten
3. Produkte aus Kundensicht vergleichen
4. Produktmerkmale ermitteln
5. Optimierungsrichtung festlegen
6. Beziehungsmatrix erstellen
7. Technische Wechselwirkungen bestimmen
8. Technische Schwierigkeiten bewerten
9. Zielwerte festlegen
10. Produkte aus technischer Sicht vergleichen
11. Technische Bedeutung bewerten



V & V – Abgleich Kunde vs. Entwickler

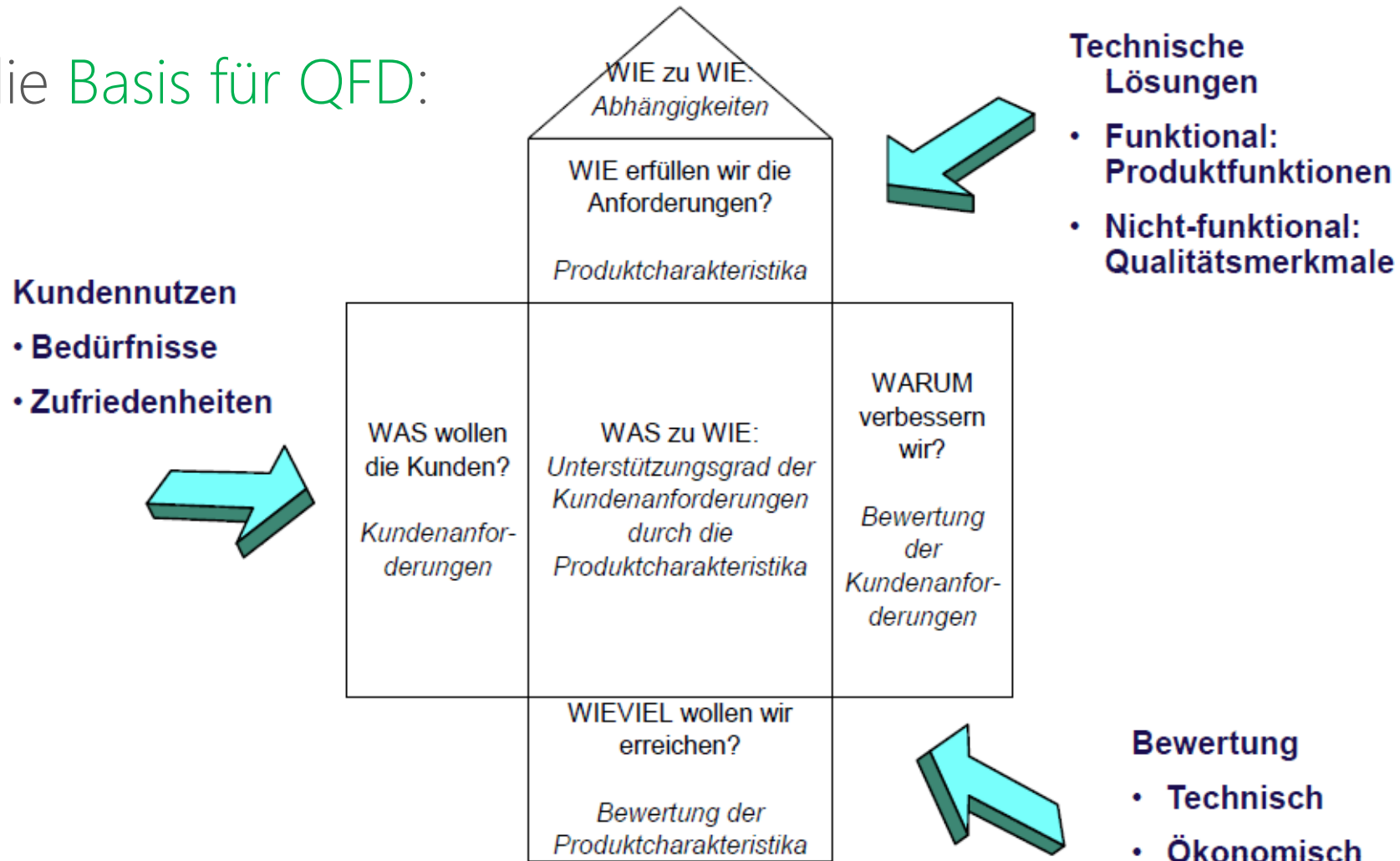
4 Möglichkeiten, ein Produkt zu gestalten:



V & V nötig!

V & V – House of Quality (HoQ) (I)

HoQ ist die Basis für QFD:

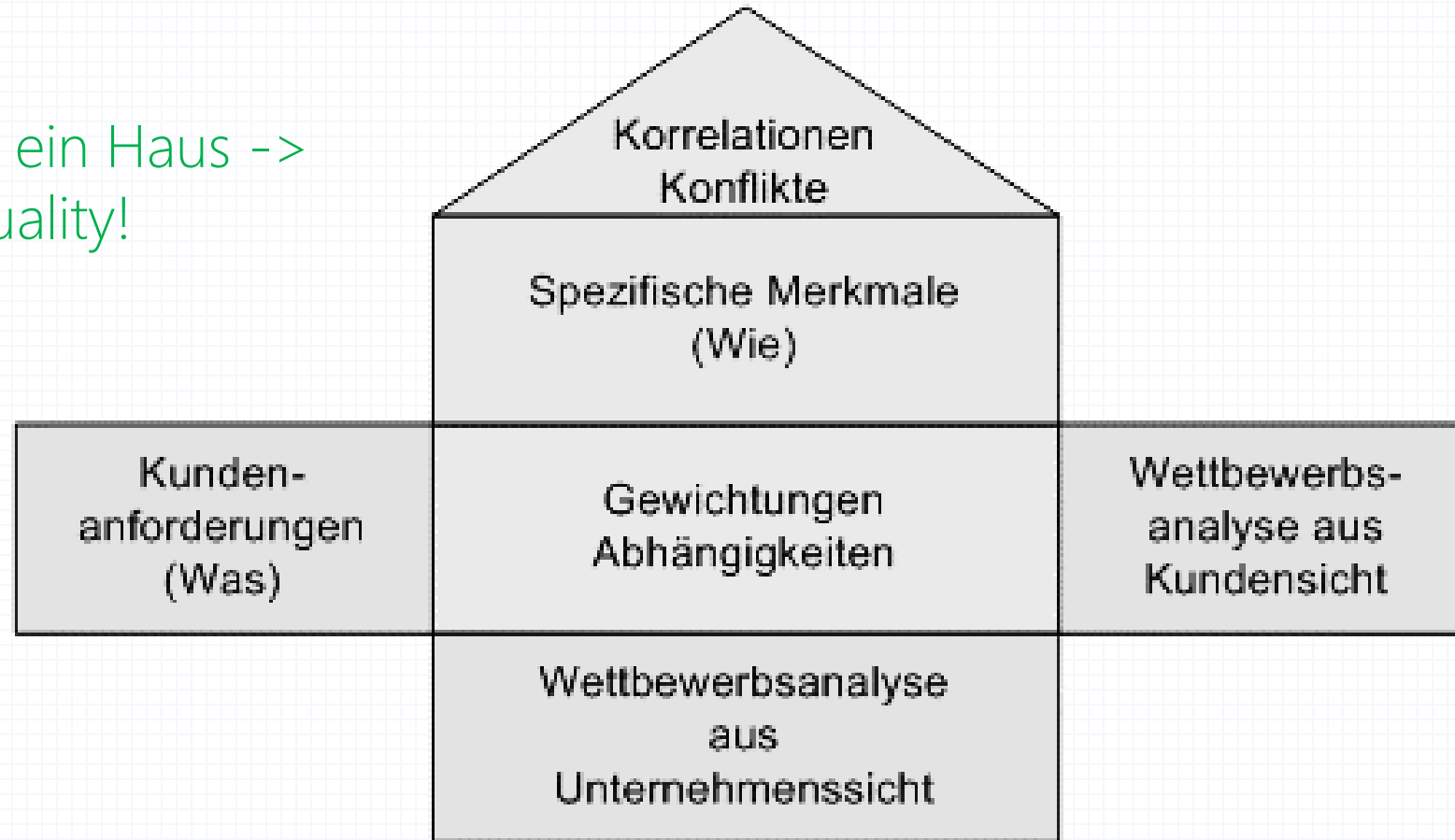


(Quelle: G. Herzwurm, BW-Institut, Univ. Stuttgart)

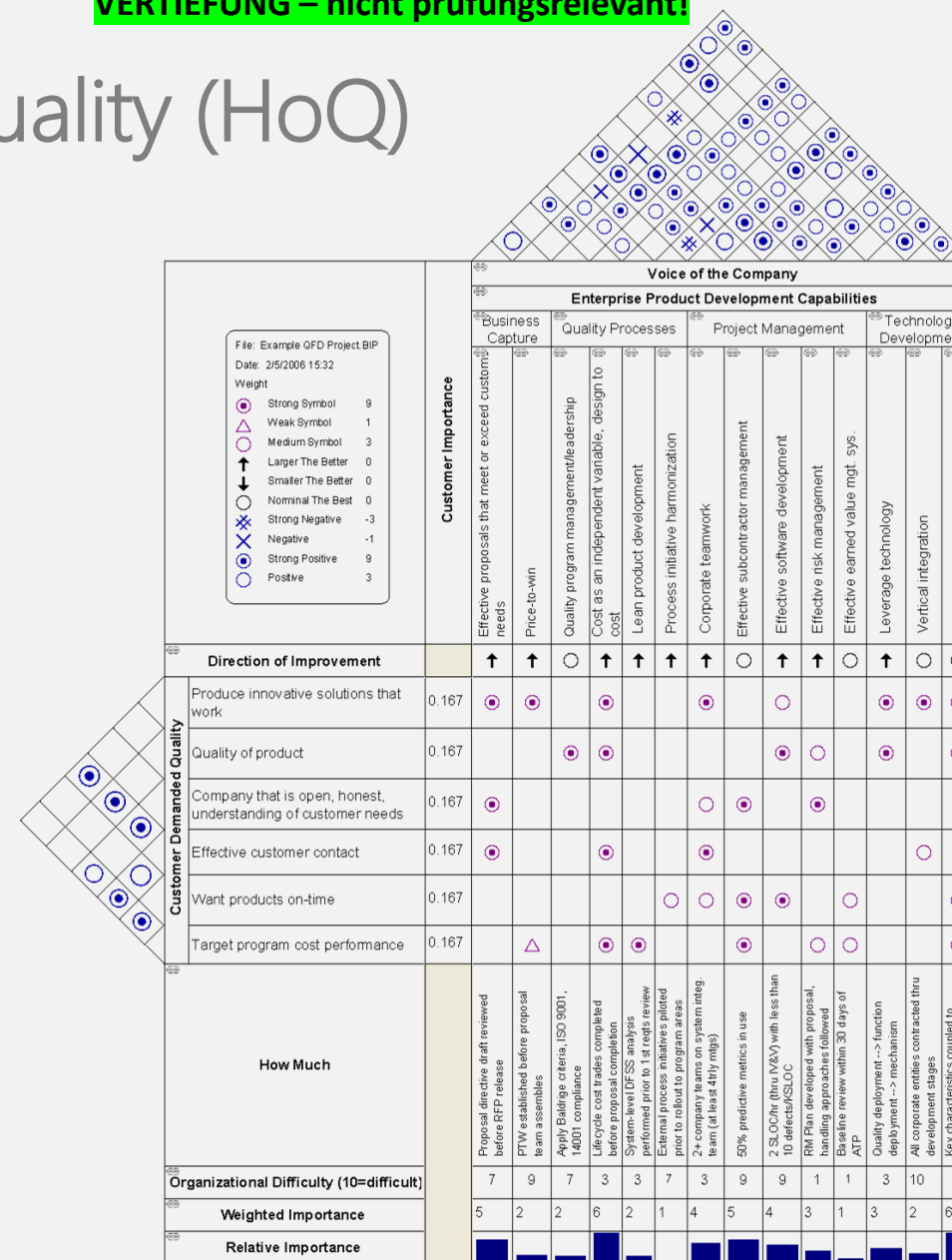
V & V – House of Quality (HoQ) (II)

Analyse der Interessenkonflikte mit HoQ :

Aussehen wie ein Haus ->
„House“ of Quality!



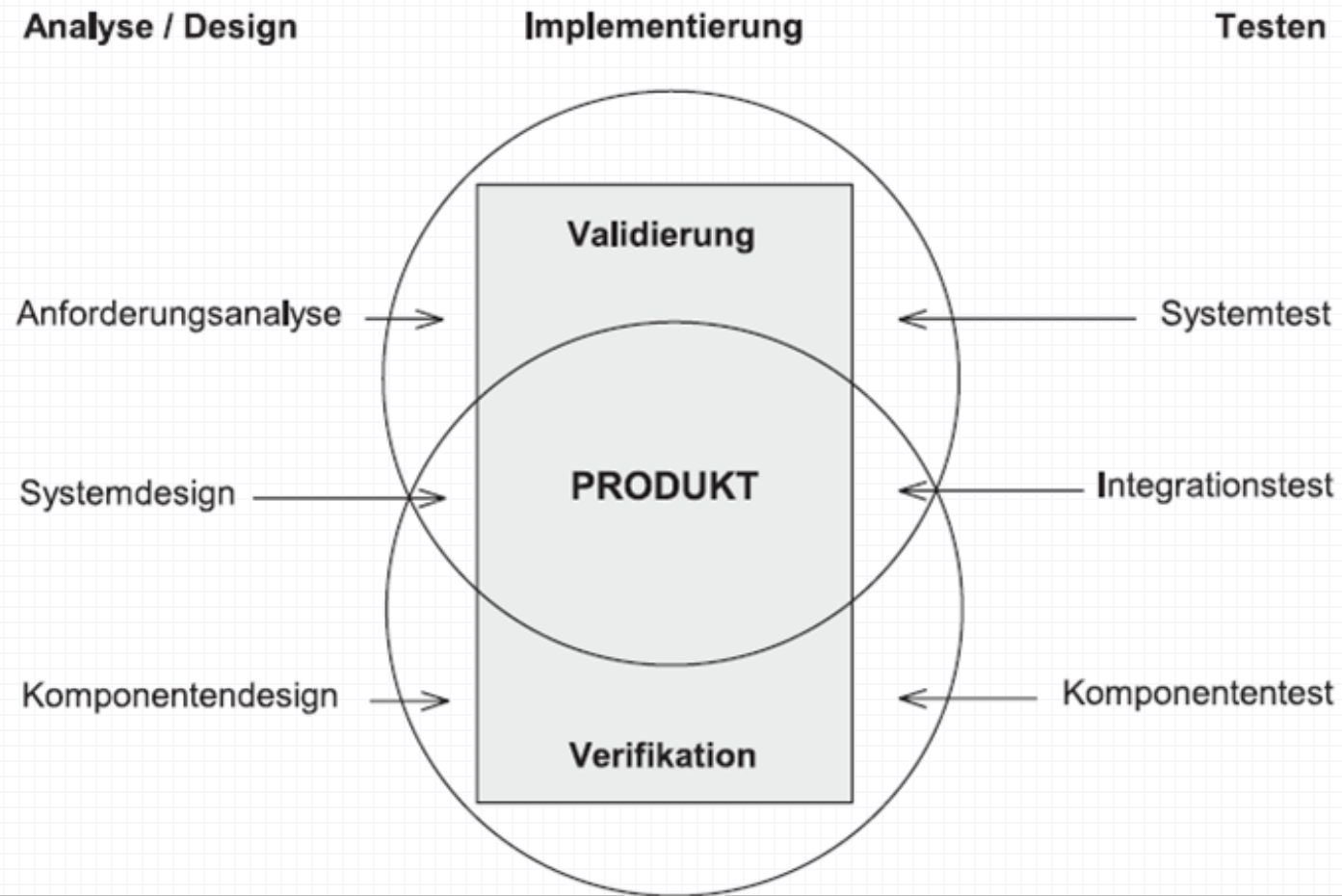
Beispiel: House of Quality (HoQ)



(Grafik: Wikipedia, free-to-use)

Techniken – Verifikation und Validierung („V & V“)

Zusammenhang **Verifikation** & **Validierung** (in USA: Testen = V&V):



-> Produkt muss verifiziert
UND validiert werden!

Techniken – Verifikation

Lastenheft, Pflichtenheft, ...

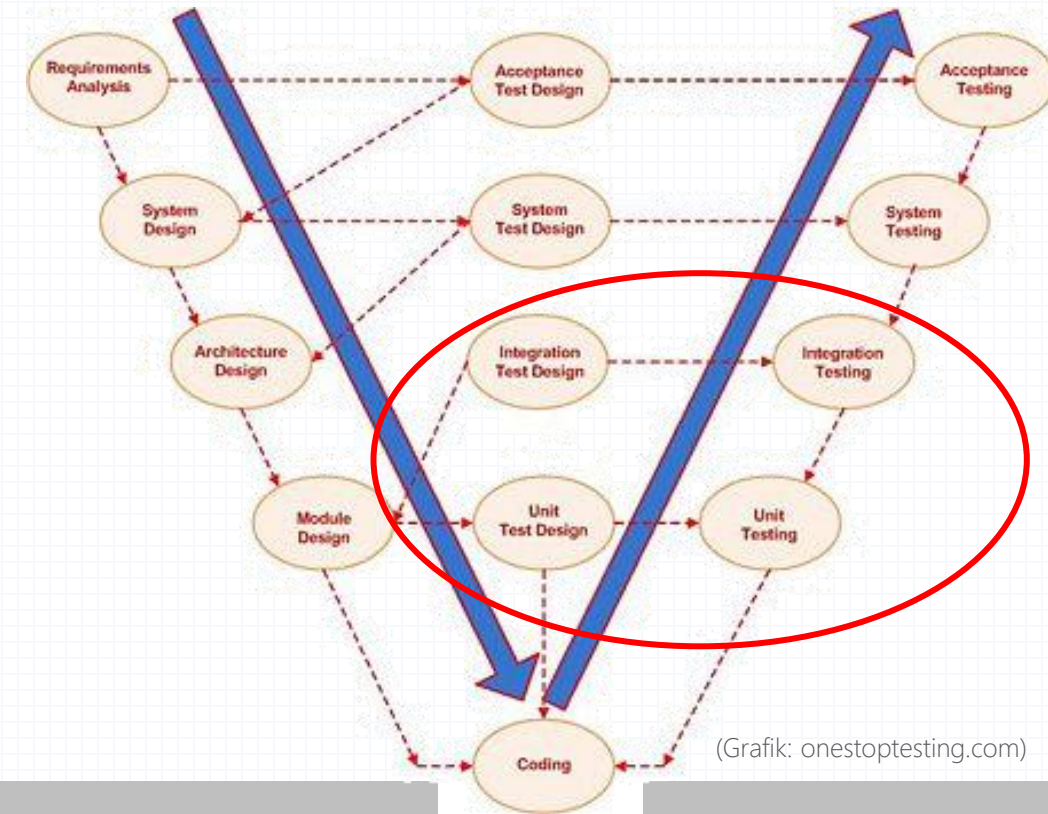
Testen der Funktionalität („Haben wir das Produkt richtig erstellt?“)

-> Überprüfung, ob ein Softwareprodukt seiner Spezifikation entspricht („**Korrektheit**“; Boehm)

daher auch „**Funktionalitätstest**“

Techniken:

- Einzeltest („Unit Test“)
- Modultest, Programmtest
- Integrationstest (Testen des Zusammenbaus)



(Grafik: onestoptesting.com)

Techniken – Validierung

engl. „End-to-End Test“

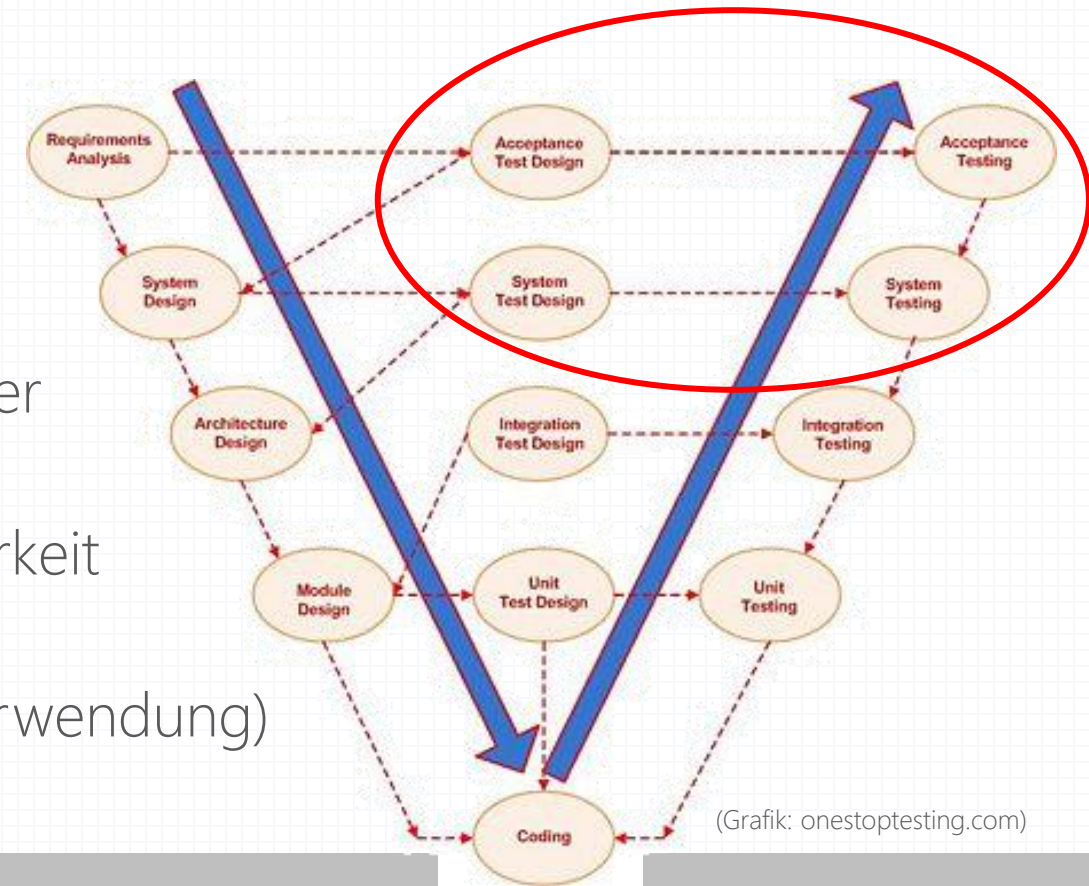
Testen des Gesamtsystems („Haben wir das richtige Produkt erstellt?“)

-> Überprüfung, ob ein Softwareprodukt den realen Anforderungen genügt („**Brauchbarkeit**“; Boehm)

daher auch „**Systemtest**“, „**Gesamttest**“

Techniken:

- Subsystemtest (Testen weitgehend unabhängiger Subsysteme und deren Schnittstellen)
- Systemtest, Integritätstest (Testen auf Einsetzbarkeit und Brauchbarkeit beim Anwender)
- (Benutzer-)Akzeptanztest (Testen der realen Verwendung)
- Zertifizierung (wenn gewünscht / erforderlich)



(Grafik: onestoptesting.com)

Testen Werkzeuge

Testen – Werkzeuge

Derzeit oft im **Open-Source-Bereich** verfügbar.

Grund: Projekte unterscheiden sich in späteren Phasen (wie dem Testen) stärker voneinander -> Markt schwierig!



Werkzeuge zur Testunterstützung:

Beispiel: <http://www.fitnesse.org/>

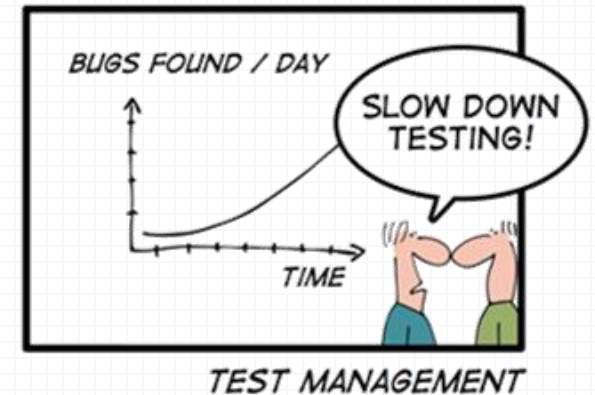
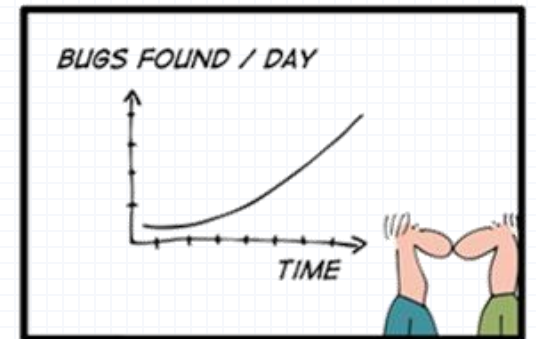
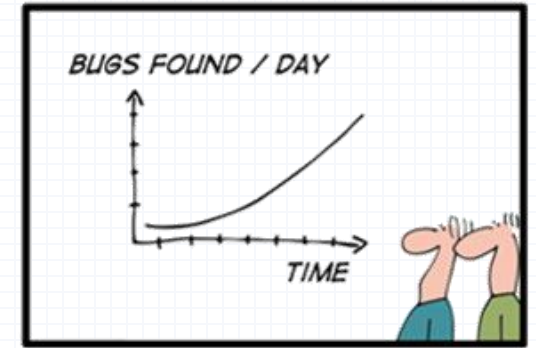
- Dateivergleichswerkzeuge
- Formatübersetzer
- Bildschirmspeicherprogramme
- Monitorprogramme
- Testrahmensysteme
- „Capture & Replay“-Systeme
- Fehlermeldesysteme
- Diagnoseprogramme
- Standardüberwachungsprogramme

Testen Ergebnisse

Testen – Ergebnisse

Ergebnisse

- Testplan
- Testsuite
- Testliste
- Fehlerbericht
- Fehlerlogbuch



Ergebnisse – Testplan (I)

Testplan = „Beschreibung von Aufgabenumfang, Vorgehensweise, Ressourcen und Ablauf der beabsichtigten Testaktivitäten“ [ANSI/IEEE]

Bedeutung ist oft Auftraggeber nicht klar!

Erstellung **benötigt viel Zeit:**

- parallel zu Design und Implementierung durchführen
- auf Pflichtenheft aufbauen
- laufend aktualisieren

(Grafik: www.firsttimequality.com)

Ergebnisse – Testplan (II)

Inhalte:

- Beschreibung von Produktziel und Testziel
- Festlegung des Testumfangs (auch was nicht getestet wird!)
- Auflistung der abgedeckten / nicht abgedeckten Risiken
- Beschreibung von Teststrategie, -ansatz und -kriterien
- Beschreibung der Testumgebung
- Planung des Ablaufs inkl. benötigter Ressourcen
- Festlegung der zu liefernden Testdokumentation

INSPECTION TEST PLAN AND LOG

CONTRACT NUMBER [Contract Name] PROJECT NAME [Project Name] CONTRACTOR [Company Name]

Item	Spec #	Specifications Section	Subsection	Inspections & Tests Required	Frequency	Inspection-Test By (All tests verified by Supervisor and/or QC Manager)	Date Completed	Date Forwarded To Contr. Off.	Remarks
1.									
2.									
3.									
4.									
5.									
6.									
7.									
8.									
9.									
10.									
11.									
12.									
13.									
14.									
15.									
16.									
17.									
18.									
19.									
20.									
21.									
22.									
23.									
24.									

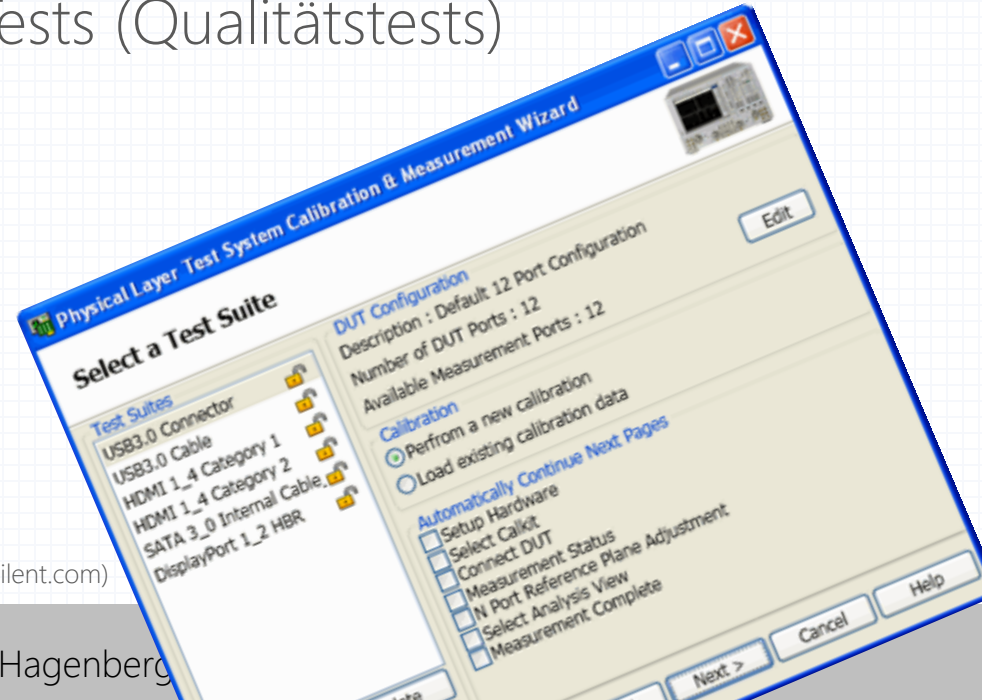
(Grafik: www.firsttimequality.com)

Ergebnisse – Testsuite

Testsuite = „nach Testklassen geordnetes Verzeichnis von Testfällen“
[ANSI/IEEE]

Einteilung in

- funktionale Tests (Konformanztests, Nonkonformanztests)
- nichtfunktionale Tests (Qualitätstests)



(Grafik: www.agilent.com)



Beispiel/Übung: Erstellen Sie einen Betatestplan + eine (kleine) Betatestsuite für das LEGO-Projekt

-> Übung (schriftlich)

Ergebnisse – Testliste

angefertigt für persönlichen oder entwicklerinternen Gebrauch,
erstellt aus dem Pflichtenheft sowie aus Benutzer- / Systemdokumentation

Beispiele für Inhalte von Testlisten:

- Funktionalitäten
- Fehlermeldungen
- im Programm enthaltene Dialoge (Eingabe, Ausgabe, Dateien)
- gelieferte Dokumente
- zum Betrieb notwendige Hardware und Software
- vom System unterstützte Hardware und Software

Ergebnisse – Fehlerbericht

Fehlerbericht = standardisierte Beschreibung eines entdeckten Fehlers mit Angaben zur Fehlerbehebung

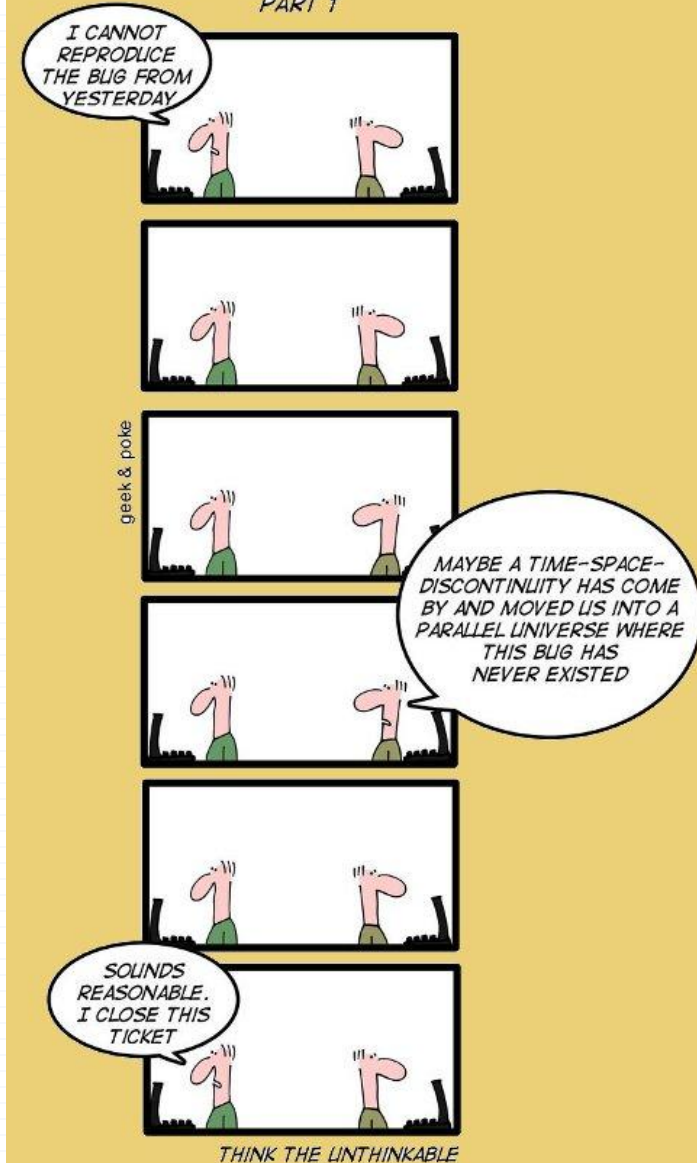
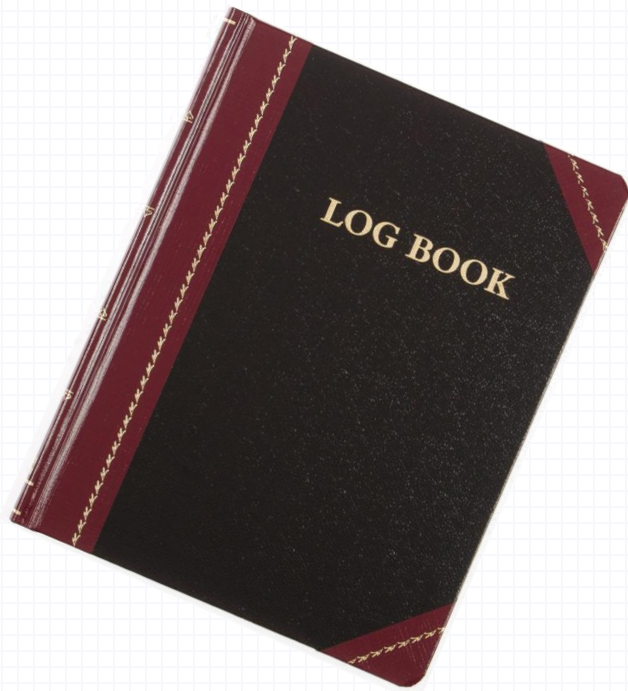
Fehler muss reproduzierbar sein!

Fehlerbericht immer sofort beim Auftreten eines Fehlers schreiben (Fehlerverhalten „noch auf dem Bildschirm sichtbar“)!



Ergebnisse – Fehlerlogbuch

Fehlerlogbuch = zeitlich geordnete Auflistung aller gefundenen Fehler inklusive ihrer Behebung (auch: **Testlogbuch**)



PROJEKT ENGINEERING

Testen

Herwig Mayr

Fakultät für Informatik, Kommunikation und Medien
Fachhochschule OÖ, Hagenberg